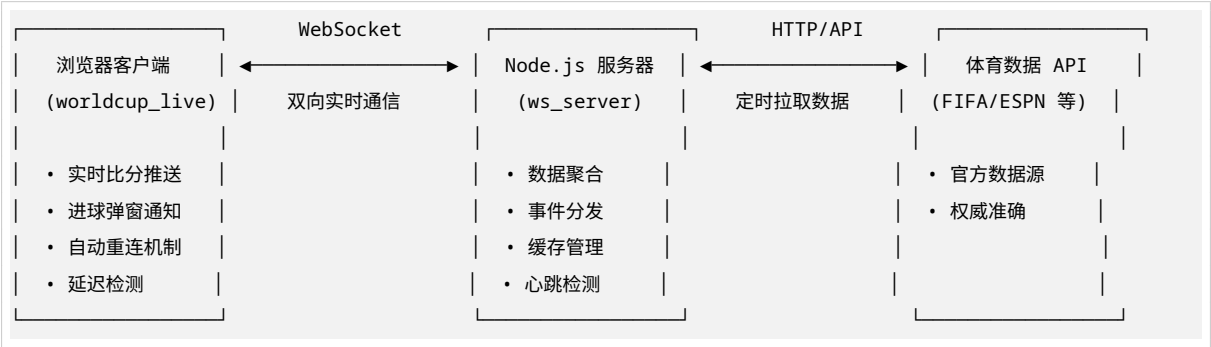


🏆 2026 世界杯 WebSocket 实时推送系统

架构概览



📁 文件说明

文件	用途	大小
ws_server.js	Node.js WebSocket 服务器	7.8 KB
worldcup_live_ws.html	前端实时页面	20 KB

🚀 快速启动

步骤 1: 安装依赖

```
# 创建项目目录
mkdir worldcup-live && cd worldcup-live

# 初始化 npm
npm init -y

# 安装 WebSocket 库
npm install ws

# 可选: 安装 Redis (用于数据缓存)
npm install redis
```

步骤 2: 启动服务器

```
# 将 ws_server.js 放入项目目录
node ws_server.js
```

输出:

```
🚀 WebSocket Server running on ws://localhost:3000
📡 HTTP API running on http://localhost:3001
🏆 World Cup 2026 Live Server Started
Features:
- WebSocket: ws://localhost:3000
- HTTP API: http://localhost:3001
- Auto-updates every 5s
```

- Real-time goal notifications

步骤 3: 打开前端页面

```
# 方式 1: 直接打开 HTML 文件 (双击)
# 方式 2: 使用本地服务器
npx serve .

# 然后访问 http://localhost:5000/worldcup_live_ws.html
```

🚀 WebSocket 协议说明

客户端 → 服务器

```
// 订阅频道
{ "action": "subscribe", "channels": ["all"] }

// 获取特定比赛
{ "action": "get_match", "matchId": "m001" }

// 获取积分榜
{ "action": "get_standings" }

// 心跳检测
{ "action": "ping", "time": 1699123456789 }
```

服务器 → 客户端

```
// 欢迎消息
{ "type": "welcome", "clientId": "xxx", "message": "Connected" }

// 初始数据
{ "type": "initial_data", "data": [...] }

// 状态更新 (每 5 秒)
{ "type": "status_update", "data": [...], "timestamp": "2026-06-22T12:00:00Z" }

// 进球通知 (实时推送)
{
  "type": "goal",
  "matchId": "m001",
  "team": "Argentina",
  "player": "Messi",
  "minute": 78,
  "newScore": "3-0"
}

// 比赛结束
{ "type": "match_end", "matchId": "m001", "finalScore": "3-0" }

// 心跳
```

```
{ "type": "heartbeat", "time": 1699123456789 }
```

部署到云端

方案 A: Render (免费)

1. 注册 render.com
2. 创建 Web Service
3. 上传代码, 设置启动命令: `node ws_server.js`
4. 获取 WebSocket URL: `wss://your-app.onrender.com`

方案 B: Railway (免费)

1. 注册 railway.app
2. 从 GitHub 导入项目
3. 设置环境变量 `PORT=3000`
4. 自动部署, 获取 URL

方案 C: 自有服务器 (VPS)

```
# 使用 PM2 进程管理
npm install -g pm2
pm2 start ws_server.js --name "worldcup-ws"
pm2 save
pm2 startup

# 使用 Nginx 反向代理 + SSL
server {
    listen 443 ssl;
    server_name wc.yourdomain.com;

    location / {
        proxy_pass http://localhost:3000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_set_header Host $host;
    }
}
```

接入真实数据源

当前 `ws_server.js` 使用模拟数据。接入真实 API:

```
// 在 ws_server.js 中替换 generateMockData()

async function fetchFromFIFA() {
    const response = await fetch('https://api.fifa.com/wc2026/matches', {
        headers: { 'Authorization': 'Bearer YOUR_API_KEY' }
    });
    return await response.json();
}
```

```
}

async function fetchFromESPN() {
  const response = await fetch('https://site.api.espn.com/apis/site/v2/sports/soccer/fifa.world/
    ↪ scoreboard');
  return await response.json();
}

// 定时拉取
setInterval(async () => {
  try {
    const realData = await fetchFromESPN();
    // 转换为内部格式
    liveData.matches = transformData(realData);
    broadcast({ type: 'status_update', data: liveData.matches });
  } catch (e) {
    console.error('API fetch failed:', e);
  }
}, 30000); // 每 30 秒拉取一次
```

 性能优化

优化项	方法	效果
连接数	使用 Redis 存储共享状态	支持 10 万 + 并发
消息大小	只推送变更数据 (delta)	减少 90% 带宽
重连	指数退避算法	避免服务器雪崩
压缩	启用 WebSocket permessage-deflate	减少 70% 流量

 安全建议

- 1. **WSS 而非 WS**: 生产环境必须使用 wss:// (SSL)
- 2. **Rate Limiting**: 限制每个 IP 的连接数和消息频率
- 3. **CORS 白名单**: 只允许特定域名访问
- 4. **Token 验证**: 连接时验证 JWT token

 客户端特性

-  自动连接 / 断开 / 重连
-  进球弹窗通知 (5 秒自动消失)
-  实时事件流 (最新 20 条)
-  连接延迟检测
-  浅色/深色主题切换
-  响应式设计 (移动端友好)